



Html Notes by Coding Age



INDEX

Day 1: HTML Fundamentals – Tags, Attributes & Structure	3
1. What is HTML?	3
2. What are Tags in HTML?	3
3. What are Attributes in HTML?	4
4. What are Sub-Tags in HTML?	5
5. Boilerplate Code of HTML	6
6. Tags in HTML(<h1>, <p>, , , <i>,etc.)	7
Day 2: HTML Tags – Common Tags and Their Uses	12
1. Structural Tags (<div>,)	15
Day 3: HTML Tags – Important Tags and Their Usage	16
1. Deprecated Tags (, <marquee>, etc.)	16
2. Links and Image Tags (<a>,)	18
Day 4: HTML Lists and Tables	20
1. Unordered and Ordered Lists (, ,)	20
2. Table Structure Tags (<table>, <tr>, <td>, <th>)	22

Day 5: HTML Multimedia and Structural Tags	25
1. Media Tags (<video>, <audio>)	25
2. Layout Tags (<iframe>, <header>, <main>, <footer>)	26
Day 6: HTML Form Elements and Employee Management System	28
1. Form Tags and Input Types (<form>, <input>, <textarea>, <select>)	29
2. Full Employee Management System Form	33
Day 7–10: HTML Project	35
1. Amazon Clone Project Overview	35

Day 1 : HTML Fundamentals – Tags, Attributes & Structure

✅ 1. What is HTML?

🔍 What:

- **HTML** stands for **HyperText Markup Language**.
- It is the **basic language used to create webpages**.
- **HyperText** = Text with links (called **hyperlinks**) that can go to other pages or parts of the same page.
- **Markup Language** = Uses **tags** to tell the browser how to display content (headings, paragraphs, images, etc.).

📖 Example Tags:

```
<h1>This is a heading</h1>
<p>This is a paragraph</p>

```

🧠 Real-Life Analogy:

| HTML is like the structure or skeleton of a house:

- HTML = bricks, walls, rooms (structure).
- CSS = paint, decorations, design (style).
- JavaScript = switches, fans, doors (functionality).

✅ 2. What are Tags in HTML?

🔍 What:

- Tags are **HTML commands** written inside **angle brackets** like `<tag>`.
- They **define and organize** elements on a webpage.

👉 Types:

- **Paired Tags:** Have both opening and closing.

```
<p>This is a paragraph</p>
```

- **Self-Closing Tags:** Don't need closing tags.

```
<br> <!-- Line Break -->  

```

🧠 Real-Life Analogy:

Tags are like labels or containers:

- A `<p>` tag says: "This text is a paragraph".
- A `<h1>` tag says: "This is a heading".

📌 Remember:

- Tags are inside `< >`
- Most tags come in pairs: `<tag>Content</tag>`
- Some are self-closing: `<br>`, `<img>`, etc.

✅ 3. What are Attributes in HTML?

🔍 What:

- **Attributes** give **extra information** about an element.
- Always written **inside the opening tag**.
- Format: `attribute="value"`

👉 Examples:

```
<a href="https://google.com">Google</a>  

```

📖 Common Attributes:

- `href` : link location
- `src` : image source
- `alt` : alternate text for image
- `class` : CSS class
- `id` : unique identifier
- `style` : inline CSS

Real-Life Analogy:

Think of an HTML tag like a car, and the attributes are its features:

```
<car color="red" model="SUV">
```

✅ Key Points:

- Attributes go inside the **opening tag**
- You can use **multiple attributes** in one tag
- They add **functionality and customization**

✅ 4. What are Sub-Tags in HTML?

What:

- **Sub-tags** are **tags nested inside other tags**.
- They create a **parent-child relationship** between elements.

Example:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>
```

Here, `<ul>` is the **parent**, and `<li>` are **sub-tags (children)**.

🧠 Real-Life Analogy:

| Like a folder with files:

- Folder = `<ul>` (unordered list)
- Files inside = `<li>` (items)

✅ Tips:

- Sub-tags go **inside** parent tags
 - Use **proper indentation** for clarity
-

✅ 5. Boilerplate Code of HTML

This is the **default structure** every HTML file should start with:

🧱 Boilerplate Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <!-- Webpage content goes here →
</body>
</html>
```

🔍 Breakdown:

1. `<!DOCTYPE html>`
 - Declares the document is **HTML5**.
 - Must be at the top.
 - Not a tag — it's an instruction.

2. `<html lang="en">`
 - Root element for the page.
 - `lang="en"` tells browser the language is English.
 3. `<head>` **Section**
 - Contains **meta** info (not shown on page):
 - Character set, title, links to CSS, scripts, etc.
 4. `<meta charset="UTF-8">`
 - Tells browser to use **UTF-8 encoding** (supports many languages).
 5. `<meta name="viewport"...>`
 - Makes your site **mobile responsive**.
 - Adjusts the size/zoom based on device screen.
 6. `<title>`
 - Sets the title of the browser tab.
 - Important for **SEO** (Google ranking).
 7. `<body>`
 - The main visible **content** of the page goes here (headings, paragraphs, images, etc.).
-

6. Tags in HTML – Detailed Explanation

1. `<h1>` to `<h6>`

What:

These tags define headings in HTML. `<h1>` is the largest (main heading), and `<h6>` is the smallest (least important heading).

Why:

Headings create a clear structure on a webpage, helping users and search engines understand the content hierarchy.

How:

- Use `<h1>` for the main title of your page (only one per page is recommended).

- Use `<h2>` to `<h6>` for subheadings or sections with decreasing importance.

Attributes:

- `id` — unique identifier for CSS/JavaScript targeting.
- `class` — group of elements for styling.
- `style` — inline CSS styles.

Example:

```
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Sub-subheading</h3>
```

2. `<p>`

What:

Defines a paragraph of text.

Why:

Paragraphs organize text content clearly on a webpage.

How:

- Paragraphs are block-level elements, meaning they start on a new line and create space before and after.

Attributes:

- `align` (deprecated) — controls text alignment (`left` , `center` , `right`). Use CSS instead (`text-align`).
- `id` , `class` , `style` for identification and styling.

Example:

```
<p align="center">This is a centered paragraph.</p>
```

3. `<i>`

What:

Italicizes text, mainly for stylistic emphasis.

Why:

Used for titles, foreign words, or stylistic emphasis without extra meaning.

How:

Wrap text inside `<i>...</i>` .

Attributes:

`id` , `class` , `style` as usual.

Example:

```
<i>This text is italicized.</i>
```

4. `<em>`

What:

Emphasizes text, usually shown in italics.

Why:

Gives semantic emphasis, useful for screen readers and accessibility.

How:

Wrap important or stressed text inside `<em>...</em>` .

Attributes:

`id` , `class` , `style` .

Difference between `<i>` and `<em>` :

- `<i>` is just visual italic styling.
- `<em>` conveys *meaning* or *importance* and improves accessibility.

Example:

```
<em>This text is emphasized.</em>
```

5. `<b>`

What:

Makes text bold.

Why:

To draw attention to text without implying special importance.

How:

Wrap the text inside `<b>...</b>` .

Attributes:

`id` , `class` , `style` .

Example:

```
<b>This text is bold.</b>
```

6. <small>**What:**

Makes text smaller than surrounding text.

Why:

Used for footnotes, disclaimers, or less important info.

How:

Wrap text inside `<small>...</small>` .

Attributes:

`id` , `class` , `style` .

Example:

```
<small>This is small text.</small>
```

7. <big>**What:**

Makes text slightly larger than surrounding text.

Why:

To highlight important points or key words.

How:

Wrap text inside `<big>...</big>` .

Attributes:

`id` , `class` , `style` .

Example:

```
<big>This is big text.</big>
```

8. `<pre>`

What:

Displays preformatted text — spaces, tabs, and line breaks are preserved exactly.

Why:

Great for showing code, poetry, or any text where formatting matters.

How:

Wrap text/code inside `<pre>...</pre>` . Avoid overly long preformatted text for readability.

Attributes:

- `id` , `class` , `style`
- `width` (not commonly used)

Example:

```
<pre>
This is preformatted text.
  Indentation and
  line breaks are preserved.
</pre>
```

9. `<br>`

What:

Creates a line break (new line) within text.

Why:

Used to break lines without starting a new paragraph.

How:

Insert `<br>` where you want a line break. It's an empty/self-closing tag (no closing tag).

Attributes:

None.

Example:

```
This is the first line.<br>
This is the second line.
```

Day 2 : HTML Tags – Common Tags and Their Uses

1. `<body>`

- **What:** Contains the main visible content of an HTML page.
- **Why:** Displays text, images, videos, etc. in the browser.
- **How:** Wrap all page content inside `<body>`.
- **Attributes:**
 - `bgcolor` (deprecated, use CSS): background color
 - `text` (deprecated, use CSS): text color
 - `link` : unvisited link color
 - `vlink` : visited link color
 - `alink` : active link color
- **Example:**

```
<body bgcolor="lightblue" text="black" link="blue" vlink="purple" alink="red">
  <h1>Welcome to My Page</h1>
</body>
```

2. `<center>` (Deprecated)

- **What:** Centers content horizontally (use CSS `text-align: center` instead).
- **How:** Wrap content inside `<center>`.
- **Attributes:**
 - `title`: shows tooltip on hover.
- **Example:**

```
<center title="Centered Content">  
  <h1>Centered Content</h1>  
</center>
```

3. `<strong>`

- **What:** Emphasizes important text, usually bold.
- **Why:** Semantically shows importance.
- **Example:**

```
<strong>This is important text.</strong>
```

4. `<mark>`

- **What:** Highlights text with a background color (usually yellow).
- **Why:** Draws attention to specific parts.
- **Example:**

```
<p>This is <mark>highlighted</mark> text.</p>
```

5. ` ` (Non-Breaking Space)

- **What:** Creates a space that will not collapse.
- **Why:** Adds extra spaces where needed.
- **Example:**

```
<p>This &nbsp; &nbsp; &nbsp; is &nbsp; &nbsp; &nbsp; spaced text.</p>
```

6. `<hr>`

- **What:** Creates a horizontal line (separator).
- **Why:** Visually separates content sections.
- **Attributes:**
 - `width` : width of line (e.g., `50%` , `200px`)
 - `size` : thickness in pixels
 - `color` (deprecated, use CSS)
 - `align` (deprecated, use CSS)
- **Example:**

```
<hr size="5" color="blue" width="50%">
```

7. `<u>` (Deprecated)

- **What:** Underlines text (use CSS `text-decoration: underline` instead).
- **Example:**

```
<u>This text is underlined.</u>
```

8. `<del>`

- **What:** Marks text as deleted with strikethrough.
- **Why:** Shows content was removed.
- **Example:**

```
<del>This text was deleted.</del>
```

9. `<strike>` (Deprecated)

- **What:** Strikethrough text (use `<del>` or CSS instead).
- **Example:**

```
<strike>This text is crossed out.</strike>
```

10. `<ins>`

- **What:** Marks text as inserted, usually underlined.
- **Why:** Highlights added content.
- **Example:**

```
<ins>This text was inserted.</ins>
```

11. `<div>`

- **What:** Defines a block-level section/division.
- **Why:** Groups elements for styling or scripting.
- **Attributes:** `id`, `class`, `style`
- **Example:**

```
<div style="background-color: lightblue;">  
  <p>This is inside a div.</p>  
</div>
```

12. `<span>`

- **What:** Inline container to group text or elements.
- **Why:** Apply styles/scripts without breaking flow.
- **Attributes:** `id`, `class`, `style`
- **Example:**

```
<p>This is a <span style="color: red;">red text</span> example.</p>
```

Day 3 : HTML Tags – Important Tags and Their Usage

1. `<font>` Tag (Deprecated)

- **What is it?**

The `<font>` tag was used in older HTML to change how text looks by setting its size, color, and font style directly in the HTML code.

- **Why was it used?**

Before CSS (Cascading Style Sheets), this was the only way to style text on web pages.

- **How to use it?**

You write text inside the `<font>` tag and add attributes like `size`, `color`, and `face` to control how the text looks.

- **Attributes:**

- `size` : Controls text size; values from 1 (smallest) to 7 (largest). For example, `<font size="4">`
- `color` : Changes text color using color names or hex codes. For example, `<font color="red">` or `<font color="#00FF00">`
- `face` : Specifies the font family like Arial, Times New Roman, etc. For example, `<font face="Arial">`

- **Example:**

```
<font size="5" color="blue" face="Arial">This text is big, blue, and Arial font.</font>
```

- **Important Note:**

This tag is outdated and **should not** be used now. Instead, use CSS to style text. For example:

```
<p style="font-size: 20px; color: blue; font-family: Arial;">This text is styled with CSS.</p>
```

2. `<sup>` Tag (Superscript)

- **What is it?**

The `<sup>` tag is used to write small text above the normal text line.

- **Why do we use it?**

For things like exponents in math (e.g., 5^2), footnote numbers in documents, or any notation that needs to appear slightly raised.

- **How to use it?**

Put the text you want to show as superscript inside the `<sup>` tag.

- **Example:**

```
<p>5<sup>2</sup> means 5 squared, which equals 25.</p>
```

The number "2" will appear a little higher than the rest of the text.

3. `<sub>` Tag (Subscript)

- **What is it?**

The `<sub>` tag writes small text slightly below the normal line.

- **Why do we use it?**

Mostly for scientific or chemical formulas like water (H_2O), where the 2 is smaller and lower.

- **How to use it?**

Wrap the text you want to be subscript inside the `<sub>` tag.

- **Example:**

```
<p>Water is H<sub>2</sub>O.</p>
```

The "2" appears below the "H".

4. `<marquee>` Tag (Deprecated)

- **What is it?**

This tag makes text or images scroll (move) across the webpage.

- **Why was it used?**

To create moving announcements or catch the user's attention with scrolling text.

- **How to use it?**

Write your text inside the `<marquee>` tag and use attributes to control scrolling direction, speed, and behavior.

- **Attributes:**

- `direction` : Which way the text moves (`left` , `right` , `up` , `down`)
- `behavior` : How the text moves:
 - `scroll` (default): moves continuously
 - `slide` : moves once and stops
 - `alternate` : moves back and forth
- `scrollamount` : Speed (higher number = faster)
- `loop` : How many times the scroll repeats (`infinite` for endless)
- `width` & `height` : Size of the marquee area

- **Example:**

```
<marquee direction="left" behavior="scroll" scrollamount="10">  
  This text scrolls from right to left!  
</marquee>
```

- **Important Note:**

This tag is **deprecated** because it is bad for accessibility and design. Use **CSS animations** for scrolling effects instead.

5. `<a>` Tag (Anchor - Hyperlink)

- **What is it?**

The `<a>` tag creates links on your webpage. These links can take users to other web pages, files, or email addresses.

- **Why do we use it?**

To connect different pages and resources, allowing easy navigation on the internet.

- **How to use it?**

Wrap the clickable text or element inside the `<a>` tag and use the `href` attribute to specify the destination.

- **Attributes:**

- `href` : The URL or address where the link goes. For example, `"https://www.google.com"`
- `target` : Where to open the link:
 - `_self` (default): opens in the same tab
 - `_blank` : opens in a new tab
- `title` : A tooltip that shows when you hover over the link

- **Example:**

```
<a href="https://www.google.com" target="_blank" title="Go to Google">Google</a>
```

This link opens Google in a new tab, and on hover, it shows "Go to Google".

6. `<img>` Tag (Image)

- **What is it?**

The `<img>` tag is used to display images on a webpage.

- **Why do we use it?**

Images make webpages more attractive and help explain ideas visually.

- **How to use it?**

Use the `<img>` tag with the `src` attribute to point to the image file location.

- **Attributes:**

- `src` (required): Image URL or file path
- `alt` : Text description for screen readers or if the image fails to load
- `width` & `height` : Set image size (pixels or %)
- `title` : Tooltip when hovering over the image
- `loading` : Controls image loading (`lazy` or `eager`)

- `crossorigin` : Controls security settings for images loaded from other websites

- **Example:**

```

```

This shows an image called `flower.jpg`, gives a description for accessibility, sets its size, and loads it only when needed.

Day 4: HTML Lists and Tables

1. `<ul>` (Unordered List)

- **What:**

Defines an unordered list — list items appear with bullet points by default.

- **Why:**

Used for grouping items where order doesn't matter (e.g., features, grocery list).

- **How:**

Contains one or more `<li>` (list item) elements inside.

- **Attributes:**

- `type` — Bullet style:
 - `disc` (default, solid circle)
 - `circle` (empty circle)
 - `square` (square bullet)
 - `none` (no bullet)
- `id`, `class`, `style` — For styling or identification.

- **Example:**

```
<ul>  
  <li>Item 1</li>  
  <li>Item 2</li>
```

```
<li>Item 3</li>
</ul>
```

2. `<li>` (List Item)

- **What:**

Defines a single item inside an ordered (`<ol>`) or unordered (`<ul>`) list.

- **Why:**

Basic building block of lists.

- **How:**

Must be wrapped inside `<ul>` or `<ol>` .

- **Example:**

```
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Oranges</li>
</ul>
```

3. `<ol>` (Ordered List)

- **What:**

Defines an ordered (numbered) list.

- **Why:**

Used when order matters, like steps in a process or ranking.

- **How:**

Contains `<li>` elements.

- **Attributes:**

- `type` — Number style:
 - `1` (numbers, default)
 - `A` (uppercase letters)
 - `a` (lowercase letters)

- `I` (uppercase Roman numerals)
- `i` (lowercase Roman numerals)
- `start` — Starting number (default 1)
- `reversed` — Displays list in reverse order.
- **Example:**

```
<ol type="A" start="2">
  <li value="5">Step 1</li>
  <li>Step 2</li>
  <li>Step 3</li>
</ol>
```

This will show list starting at letter B, but first item numbered 5.

4. `<table>`

- **What:**
Defines a table to organize data in rows and columns.
- **Why:**
To present tabular data clearly (e.g., schedules, price lists).
- **How:**
Nested tags:
 - `<tr>` for table rows
 - `<th>` for header cells
 - `<td>` for data cells
 Optional:
 - `<caption>` for table title
 - `<thead>`, `<tbody>`, `<tfoot>` to group sections.
- **Attributes (mostly deprecated):**
 - `border` — Border width (use CSS instead)
 - `cellspacing` — Space between cells (use CSS)

- `cellpadding` — Space inside cells (use CSS)

- **Example:**

```
<table>
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
  </tr>
  <tr>
    <td>Row 1, Cell 1</td>
    <td>Row 1, Cell 2</td>
  </tr>
  <tr>
    <td>Row 2, Cell 1</td>
    <td>Row 2, Cell 2</td>
  </tr>
</table>
```

5. `<tr>` (Table Row)

- **What:**

Defines a row in the table.

- **Why:**

Groups cells horizontally.

- **How:**

Contains `<td>` or `<th>` cells.

- **Attributes:**

`id`, `class`, `style` for styling or identification.

- **Example:**

```
<tr>
  <td>Data 1</td>
  <td>Data 2</td>
</tr>
```

6. `<th>` (Table Header Cell)

- **What:**

Defines a header cell (bold and centered by default).

- **Why:**

Provides labels for columns or rows.

- **How:**

Use inside `<tr>` to mark headers.

- **Attributes:**

- `colspan` — Span multiple columns
- `rowspan` — Span multiple rows
- `id` , `class` , `style`

- **Example:**

```
<tr>
  <th colspan="3">Product</th>
</tr>
```

7. `<td>` (Table Data Cell)

- **What:**

Defines a normal cell with data.

- **Why:**

Displays actual table content.

- **How:**

Use inside `<tr>` rows.

- **Attributes:**

- `colspan` — Span columns
- `rowspan` — Span rows
- `id` , `class` , `style`

- **Example:**


```
<tr>
  <td>Data 1</td>
  <td>Data 2</td>
</tr>
```

Day 5: HTML Tags – Multimedia and Structural Tags

1. `<video>` (Video Tag)

What:

Used to embed videos on a webpage.

Why:

To show videos like tutorials, ads, entertainment, etc.

How:

Use `<video>` tag with one or more `<source>` tags inside for different video formats for better browser support.

Common Attributes:

- `src`: video file URL (can be replaced with multiple `<source>` tags)
- `controls`: shows play, pause, volume controls
- `autoplay`: video starts automatically
- `loop`: video repeats continuously
- `muted`: video starts muted
- `poster`: image shown before video plays
- `width` & `height`: video player size

Example:

```
<video controls width="600" poster="poster.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

2. `<audio>` (Audio Tag)

What:

Used to embed audio content (music, podcasts).

Why:

To add sound to your website for better user experience.

How:

Like `<video>`, use `<audio>` with `<source>` tags for different audio formats.

Common Attributes:

- `src`: audio file URL (can use multiple `<source>` tags)
- `controls`: show play, pause, volume controls
- `autoplay`: start audio automatically
- `loop`: repeat audio continuously
- `muted`: start audio muted

Example:

```
<audio controls>
  <source src="audio.mp3" type="audio/mp3">
  <source src="audio.ogg" type="audio/ogg">
  Your browser does not support the audio tag.
</audio>
```

3. `<iframe>` (Inline Frame)

What:

Embeds another webpage or content (like YouTube video or Google Map) inside your page.

Why:

To show external content without leaving your webpage.

How:

Use the `src` attribute with the URL of the content.

Common Attributes:

- `src` (required): URL of content
- `width` & `height` : size of iframe
- `title` : description for accessibility
- `loading="lazy"` : loads iframe only when visible (better performance)

Example:

```
<iframe src="https://www.youtube.com/embed/r-7g08INMSI" width="600"
height="400" title="YouTube video" frameborder="0"></iframe>
```

4. `<header>` Tag

What:

Defines a header section for a page or section (like logo, site title, navigation links).

Why:

Organizes page into content, improves semantics & accessibility.

How:

Place it at the top of the page or section.

Example:

```
<header>
  <h1>My Website</h1>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
```

5. `<main>` Tag

What:

Specifies the main content area of a webpage directly related to the page's main purpose.

Why:

Helps screen readers & search engines identify the primary content easily.

Example:

```
<main>
  <h1>Welcome to My Website</h1>
  <p>This is the main content.</p>
  <article>
    <h2>Article 1</h2>
    <p>Details of article 1.</p>
  </article>
  <article>
    <h2>Article 2</h2>
    <p>Details of article 2.</p>
  </article>
</main>
```

6. `<footer>` Tag

What:

Defines the footer section of a webpage or a section (copyright, contact info).

Why:

Used for site information, links, or disclaimers at the bottom of the page.

Example:

```
<footer>
  <p>&copy; 2025 My Website. All rights reserved.</p>
  <a href="#privacy">Privacy Policy</a>
  <a href="#terms">Terms of Service</a>
</footer>
```

Day 6: HTML Form Elements and Employee Management System

1. `<form>` (Form Tag)

What:

The `<form>` tag creates an interactive form to collect user input.

Why:

Forms are essential for gathering data such as login credentials, feedback, or search queries, and sending them to a server.

How:

Wrap input fields and buttons inside the `<form>` tag. Use the `action` attribute to specify where to send the data.

Important Attributes:

- `action` — URL to send form data.
Example: `<form action="/submit-data">`
- `method` — HTTP method to send data (`GET` or `POST`).
Example: `<form method="POST">`
- `target` — Where to open the response (`_self` , `_blank`).
Example: `<form target="_blank">`

Example:

```
<form action="/submit" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>
  <button type="submit">Submit</button>
</form>
```

2. `<label>` (Label Tag)

What:

Associates a text label with a form control.

Why:

Improves accessibility and usability (clicking the label focuses the input).

How:

Use the `for` attribute linked to the input's `id`.

Example:

```
<label for="email">Email:</label>
<input type="email" id="email" name="email" required>
```

3. `<input>` (Input Tag)

What:

Creates various types of input fields (text, password, email, etc.).

Why:

Allows users to enter data in forms.

How:

Specify the `type` attribute for the input field kind.

Common Attributes & Types:

Type	Description	Example
text	Single-line text input	<code><input type="text" placeholder="Name"></code>
password	Password input (masked)	<code><input type="password"></code>
email	Email input with validation	<code><input type="email"></code>
number	Numeric input	<code><input type="number" min="0" max="10"></code>
checkbox	Checkbox	<code><input type="checkbox"></code>
radio	Radio buttons	<code><input type="radio" name="gender"></code>
submit	Submit button	<code><input type="submit" value="Submit"></code>

Other useful attributes:

- `name` — identifies the input for form submission.
- `value` — initial value or sent value on submission.
- `placeholder` — hint shown inside input.
- `maxlength` — max allowed characters.
- `required` — makes input mandatory.
- `readonly` — non-editable field.

Example:

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" placeholder="Enter your name" required>
</form>
```

4. `<textarea>` (Text Area)

What:

Creates multi-line text input.

Why:

Used for comments, feedback, or longer text.

Attributes:

- `rows` — visible height (number of text lines).
- `cols` — visible width (number of characters).
- `placeholder`, `maxlength`, `required`, `readonly` work similarly as in `<input>`.

Example:

```
<textarea rows="4" cols="50" placeholder="Enter your comment here..." maxlength="200"></textarea>
```

5. `<select>` (Dropdown List)

What:

Creates a dropdown menu for selecting options.

Why:

Saves space and organizes multiple choices.

Attributes:

- `name` — identifies dropdown in form submission.
- `multiple` — allows selecting multiple options.

- `size` — number of visible options.
- `required` — mandatory selection.

Example:

```
<form>
  <label for="city">Choose a city:</label>
  <select id="city" name="city" required>
    <option value="delhi">Delhi</option>
    <option value="mumbai">Mumbai</option>
    <option value="bangalore">Bangalore</option>
  </select>
</form>
```

6. `<option>` (Dropdown Option)

What:

Defines each choice inside `<select>`.

Attributes:

- `value` — value submitted when selected.
- `selected` — pre-selects the option by default.

Example:

```
<select name="car">
  <option value="bmw">BMW</option>
  <option value="audi" selected>Audi</option>
  <option value="tesla">Tesla</option>
</select>
```

7. `<button>` (Button Tag)

What:

Creates clickable buttons for form submission or JavaScript actions.

Attributes:

- `type` — `submit` (default inside form), `reset`, `button` (general use).

- `disabled` — disables the button.
- `value` — value sent when form is submitted.

Example:

```
<form>
  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
```

8. Employee Management System Form

A complete form example combining all tags:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Employee Management System</title>
</head>
<body>
  <h2>Employee Management System</h2>
  <form action="/submit-employee" method="POST">
    <table border="1" cellspacing="0" cellpadding="5">
      <tr>
        <th><label for="employeeid">Employee ID:</label></th>
        <td><input type="text" id="employeeid" name="employeeid" placeholder="Enter Employee ID" required></td>
      </tr>
      <tr>
        <th><label for="name">Name:</label></th>
        <td><input type="text" id="name" name="name" placeholder="Enter Full Name" required></td>
      </tr>
      <tr>
        <th><label for="email">Email:</label></th>
        <td><input type="email" id="email" name="email" placeholder="Enter
```

```

Email Address" required></td>
</tr>
<tr>
<th><label for="password">Password:</label></th>
<td><input type="password" id="password" name="password" place
holder="Enter Password" required></td>
</tr>
<tr>
<th><label for="address">Address:</label></th>
<td><textarea id="address" name="address" rows="3" placeholder
="Enter Address" required></textarea></td>
</tr>
<tr>
<th>Gender:</th>
<td>
<input type="radio" id="male" name="gender" value="Male" require
d>
<label for="male">Male</label><br>
<input type="radio" id="female" name="gender" value="Female">
<label for="female">Female</label><br>
<input type="radio" id="other" name="gender" value="Other">
<label for="other">Other</label>
</td>
</tr>
<tr>
<th><label for="city">City:</label></th>
<td>
<select id="city" name="city" required>
<option value="">-- Select City --</option>
<option value="Delhi">Delhi</option>
<option value="Mumbai">Mumbai</option>
<option value="Bangalore">Bangalore</option>
<option value="Chennai">Chennai</option>
<option value="Kolkata">Kolkata</option>
</select>
</td>
</tr>
<tr>

```

```
<th><label for="qualification">Qualification:</label></th>
<td><input type="text" id="qualification" name="qualification" placeholder="Enter Qualification" required></td>
</tr>
</table>
<br>
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
</body>
</html>
```

Day 7-10 : HTML Project

1. **Amazon Clone** with the following pages:

1. **Login Page**
2. **Sign-Up Page**
3. **Amazon Home Page**

This project will use only **HTML**.

<https://amazon-cloooooone.netlify.app/>

Project Description

Students will create a basic **Amazon Clone** with three interconnected pages using only HTML. They will learn about the following:

- Structuring forms for Login and Sign-Up pages.
 - Creating navigation links between pages.
 - Using lists, tables, and placeholders to replicate a home page layout.
-